



VIT[®]

BHOPAL

ROOMMATE / ROOM SWAPPING SYSTEM

Project Type: Console-Based Java Application

Student Name: KHUSHI KUMARI

Registration Number: 25BAI11248

Faculty Name: ADARSH PATEL

Academic Year: 2026–27

2. Introduction

The **Roommate / Room Swapping System** is a Java-based console application designed to make hostel room management easier and more organized. In a college hostel, students may sometimes need to change their rooms or exchange rooms with other students due to personal preferences, convenience, or other requirements. Managing such requests manually can be time-consuming for both students and hostel administrators.

This project provides a simple digital solution for managing student information, room details, and room-swapping requests. Users can add students, search for their information, view room details, and submit requests to exchange rooms. The administrator can review these requests and either accept or reject them based on the given conditions.

The application is completely **console-based**, which means it can be run directly through the command line without requiring a graphical user interface. This makes it simple to use and suitable for understanding the basic working of a management system.

Along with solving a practical hostel management problem, the project helps demonstrate important Java and Object-Oriented Programming concepts such as classes, objects, constructors, encapsulation, methods, ArrayList, loops, conditional statements, and input validation.

3. Problem Statement

Managing hostel rooms and room-swapping requests manually can become challenging, especially when the number of students increases. Keeping track of student details, room assignments, and pending requests requires proper organization and regular updates.

Students may find it difficult to get information about other students and their room assignments. At the same time, administrators need to manage multiple swap requests and ensure that every request is processed correctly.

Some of the common problems faced in manual room management are:

- Difficulty in maintaining accurate student and room information.
- Searching for student details takes additional time.
- Room-swapping requests are handled manually.
- Pending, accepted, and rejected requests may be difficult to track.
- Invalid or duplicate operations may create confusion.
- Room information needs to be updated after a successful swap.

To address these issues, there is a need for a simple and organized system that can maintain student and room details while providing a clear process for submitting and managing room-swapping requests.

4. Functional Requirements

The functional requirements describe the main operations that the Roommate / Room Swapping System should perform. These features allow users to manage student details, view room information, and handle room-swapping requests in an organized manner.

4.1 Add Student

The system allows the user to register a new student by entering details such as Student ID, Student Name, Email, and Current Room. Once the information is entered, it is stored in the system for future use.

4.2 View All Students

The user can view a list of all registered students along with their relevant information. This makes it easier to keep track of students living in the hostel.

4.3 Search Student

The system provides a search option that allows users to find a student using their Student ID. If the student is available in the system, their details are displayed. Otherwise, a suitable message informs the user that the student could not be found.

4.4 View Rooms

The system displays information about hostel rooms, including the room number, room type, and occupancy status. This helps users understand which rooms are occupied and which are available.

4.5 Request Room Swap

A student can submit a room-swapping request with another student by entering their Student IDs. The system checks whether both students exist and whether the request is valid

before creating the swap request.

4.6 View Swap Requests

The system allows users to view room-swapping requests along with details such as Request ID, participating students, and the current request status.

4.7 Process Swap Request

The administrator can review pending room-swapping requests and decide whether to accept or reject them. This provides a structured way to manage requests.

4.8 Update Room Information

When a room-swapping request is accepted, the system exchanges the room assignments of the two students. This helps keep the student and room information updated.

4.9 Input Validation

The system checks user inputs to reduce errors and prevent invalid operations. Examples include entering an invalid Student ID or Request ID, selecting an incorrect menu option, requesting a swap with oneself, or trying to process a request that has already been handled.

5. Non-functional Requirements

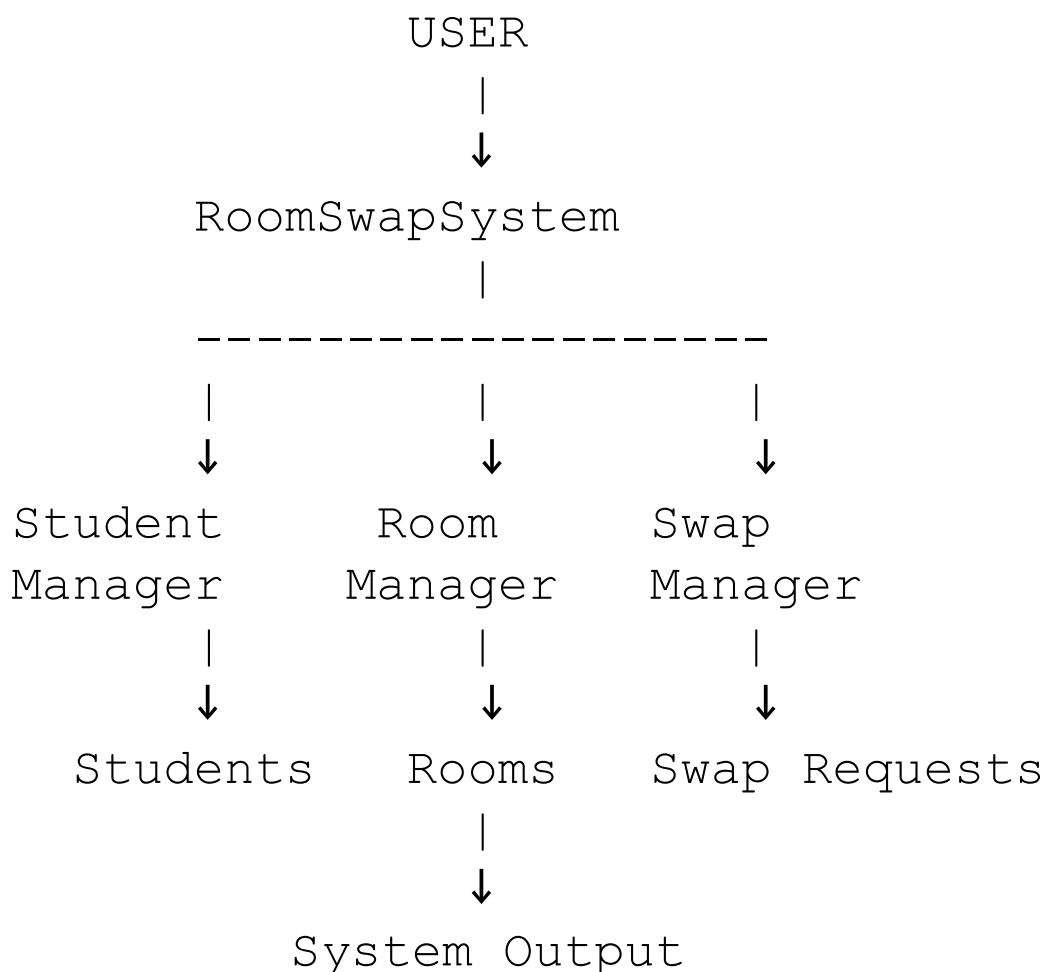
The project also satisfies the following non-functional requirements.

Requirement	Description
Usability	The system provides a simple menu-driven interface that is easy to understand.
Performance	Operations such as searching and displaying information are performed quickly for the expected project size.
Reliability	The application handles invalid inputs without unnecessarily terminating the program.
Maintainability	The project is divided into multiple classes, making the code easier to understand and modify.
Error Handling	Clear error messages are displayed when invalid operations are performed.
Scalability	The modular structure allows future features such as database connectivity to be added.
Portability	The application can run on systems that have Java installed and does not require a GUI setup.

6. System Architecture

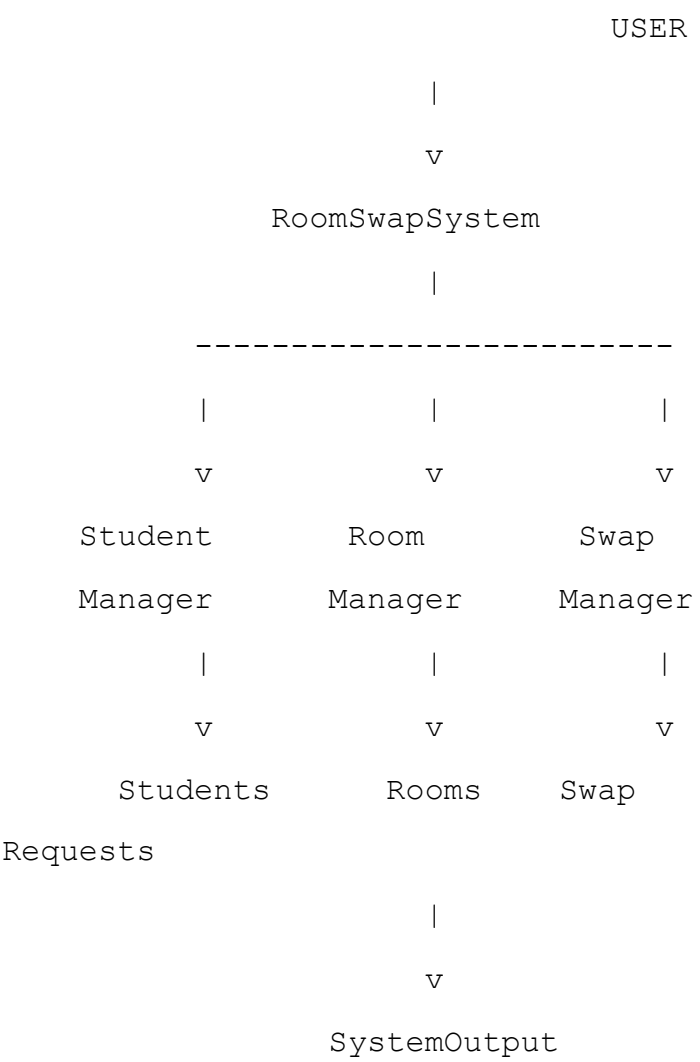
The project follows a simple modular architecture.

Architecture Flow



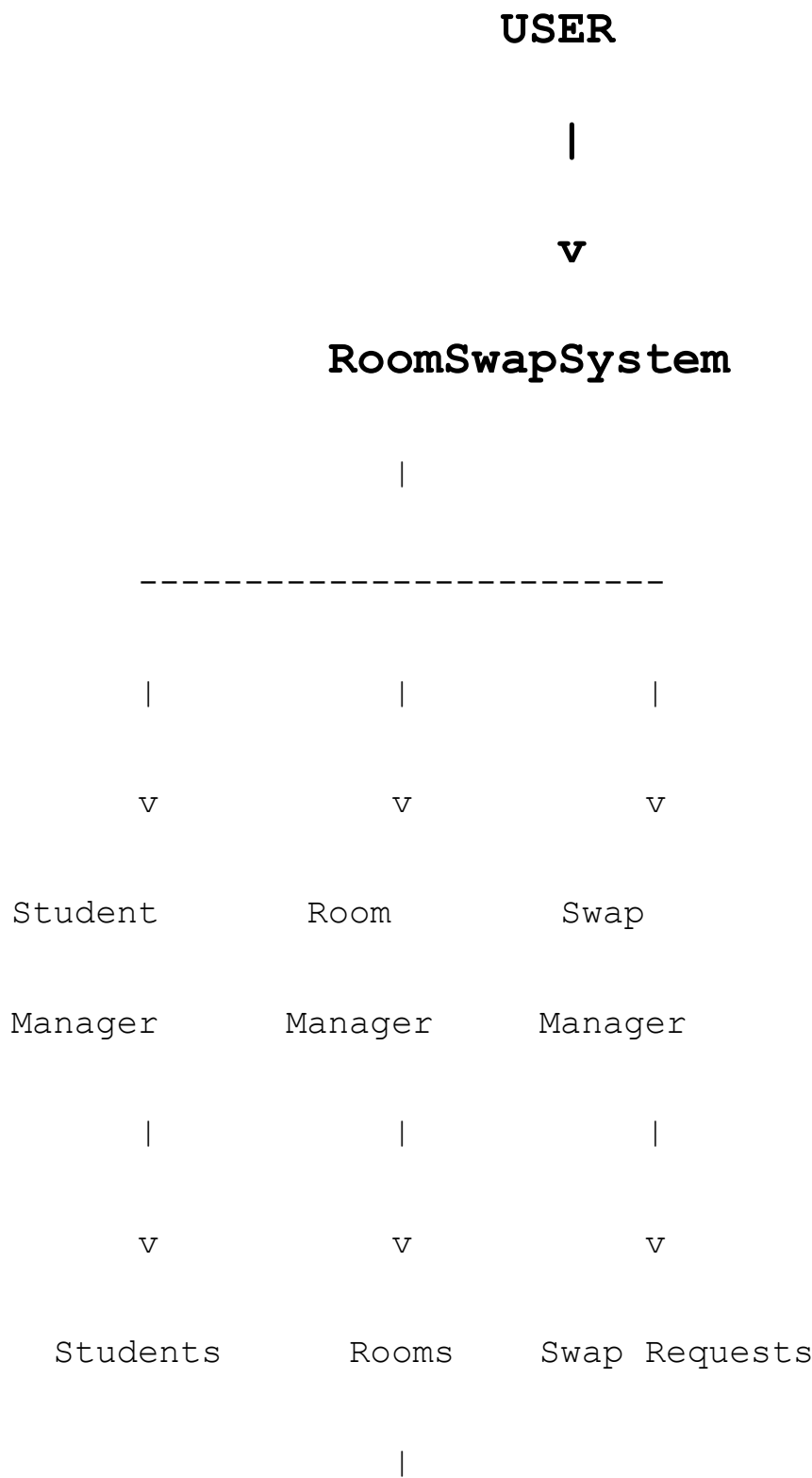
The Roommate / Room Swapping System follows a simple modular architecture. The application is divided into different components, with each component responsible for a particular task. This makes the project easier to understand and helps keep the code organized.

Architecture Flow



The Roommate / Room Swapping System follows a simple modular architecture. The application is divided into different components, with each component responsible for a particular task. This makes the project easier to understand and helps keep the code organized.

Architecture Flow



Components

1. RoomSwapSystem

This is the main class of the application. It displays the menu, accepts input from the user, and connects the different modules to perform the required operations.

2. StudentManager

The StudentManager is responsible for handling student-related activities. It allows the system to add new students, search for students, and display student information.

3. RoomManager

The RoomManager handles room-related information. It manages room details, displays room status, and helps the system keep track of room assignments.

4. SwapManager

The SwapManager manages room-swapping requests. It is responsible for creating requests, displaying pending requests, and processing requests by accepting or rejecting them.

5. Model Classes

The model classes represent the main entities used in the project:

- Student: Stores student information such as ID, name, email, and room.

- Room: Represents hostel room details and occupancy information.
- SwapRequest: Stores information about a room-swapping request and its status.
- Admin: Represents the administrator who manages and processes swap requests.

6. Data Storage

The current project uses Java ArrayList to store students, rooms, and swap requests temporarily during program execution. This provides a simple way to manage data without requiring a database.

7. Design Diagrams

7.1 Use Case Diagram

The main actors in the system are:

Student

The student can:

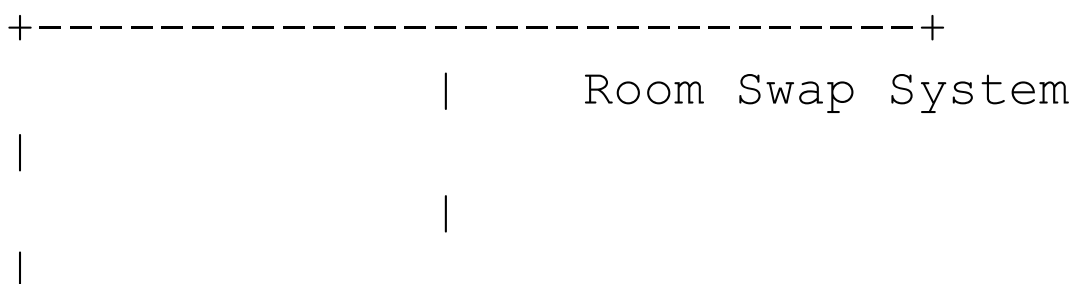
- View students
- Search students
- View rooms
- Create room swap requests
- View swap request information

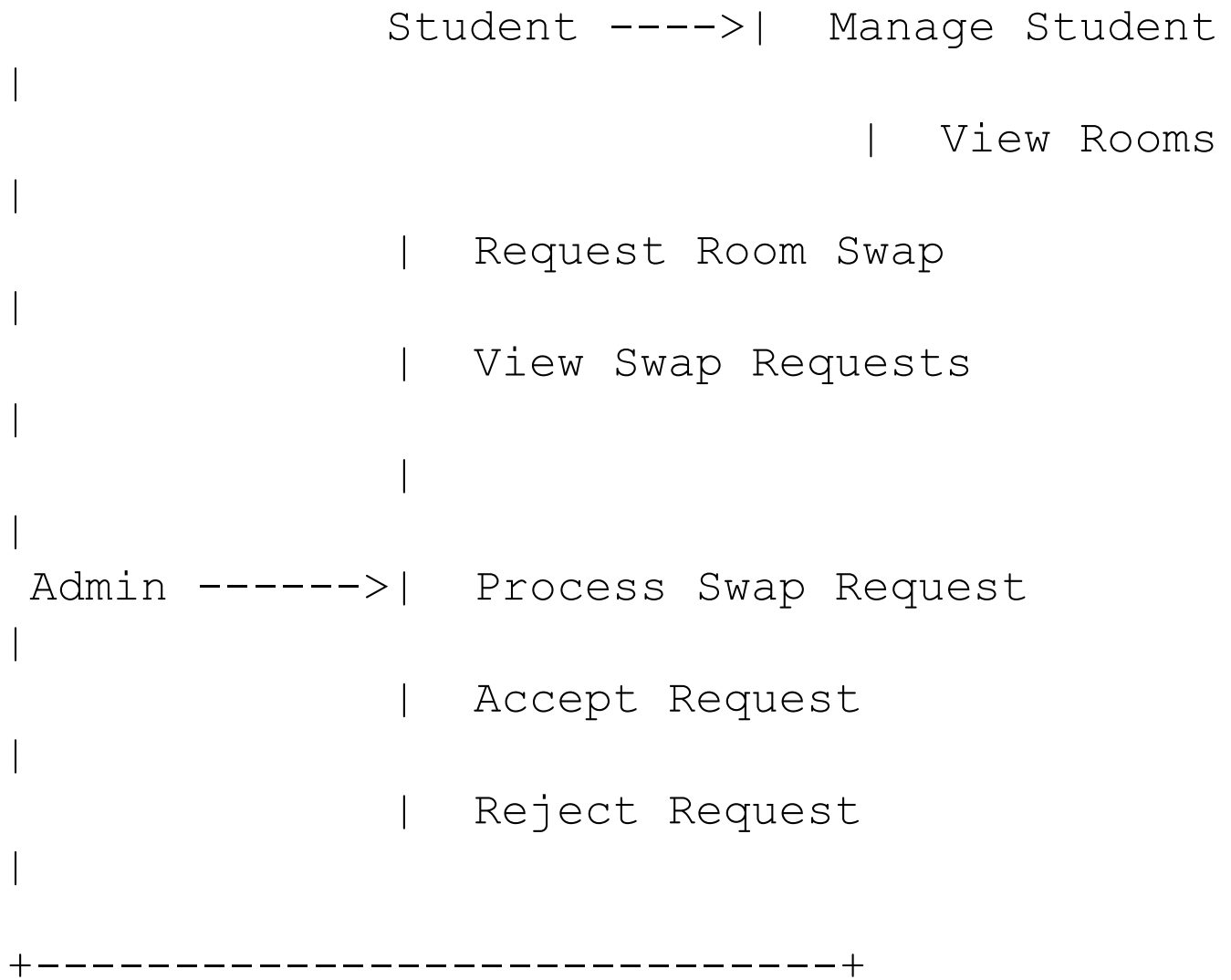
Admin

The administrator can:

- View swap requests
- Accept requests
- Reject requests
- Process room swaps

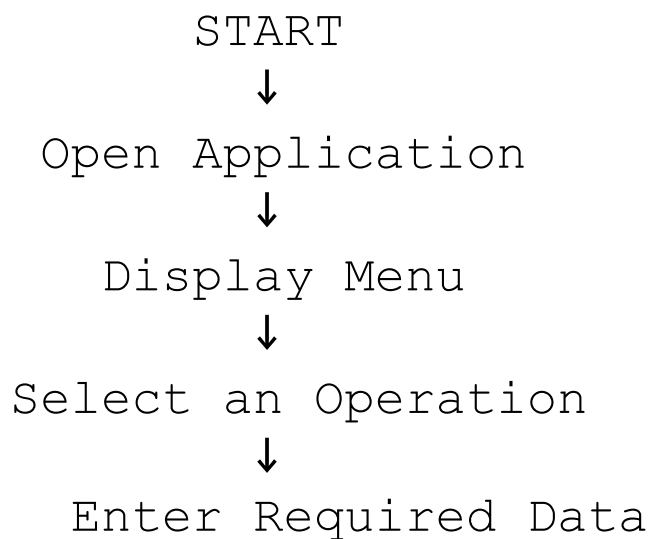
Use Case Representation

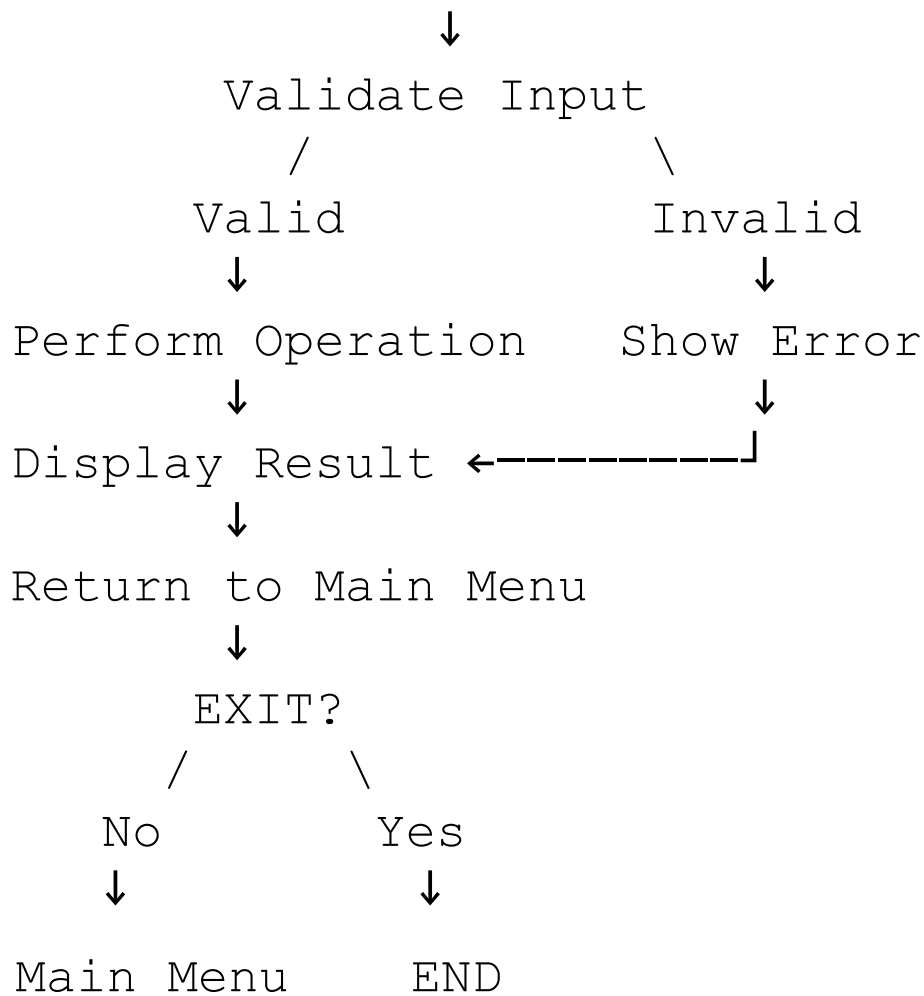




7.2 Workflow Diagram

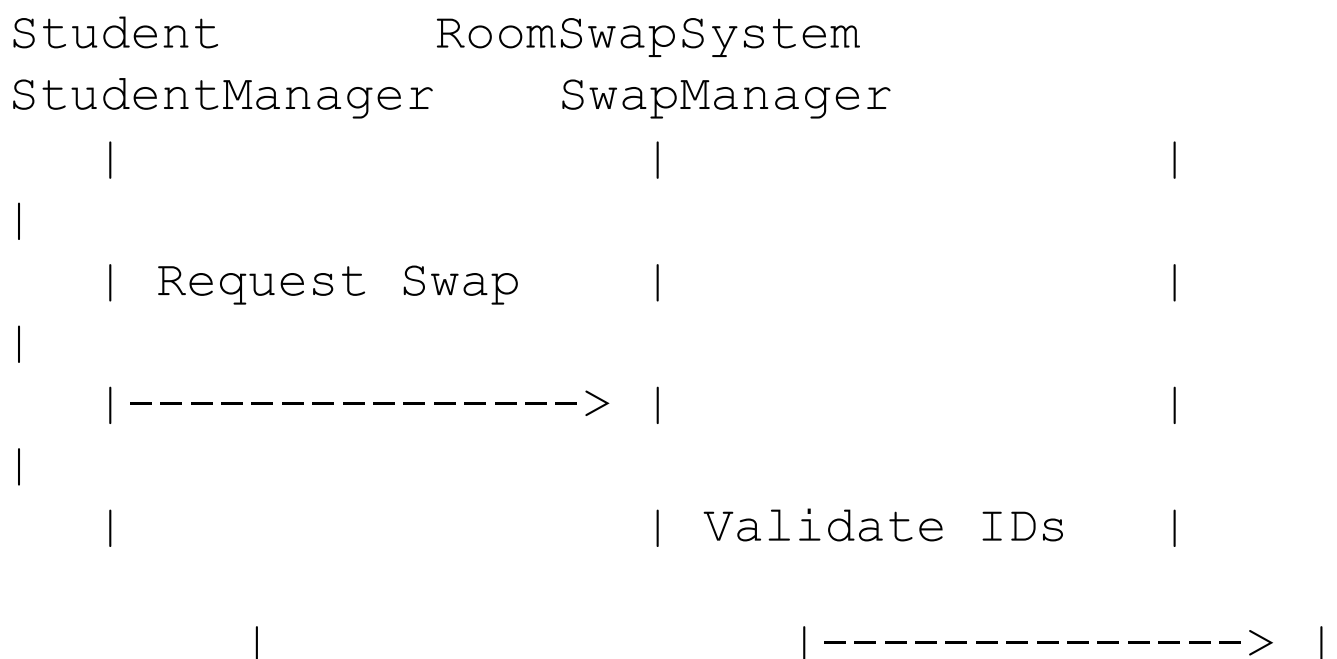
The general workflow of the system is:

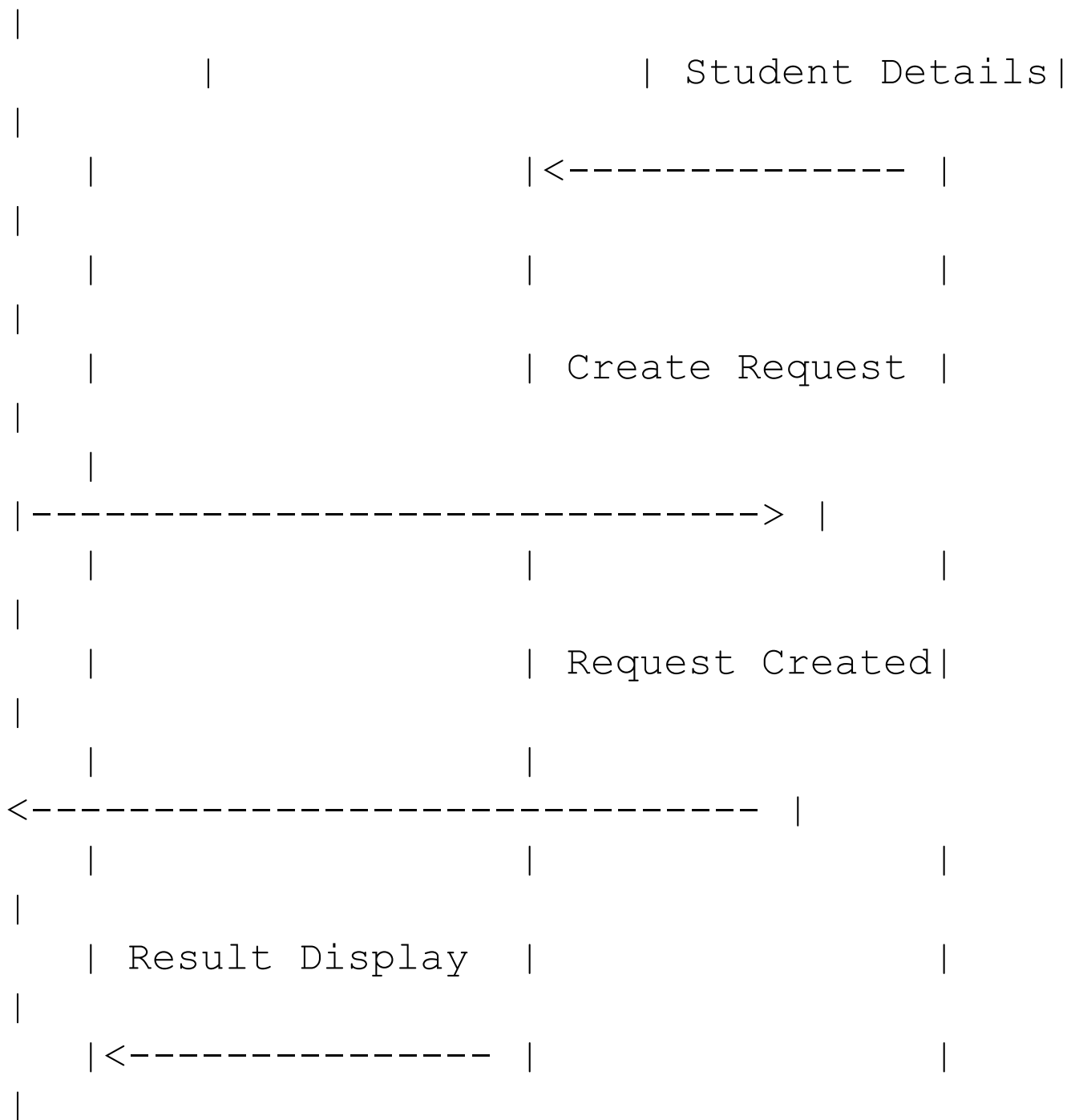




7.3 Sequence Diagram

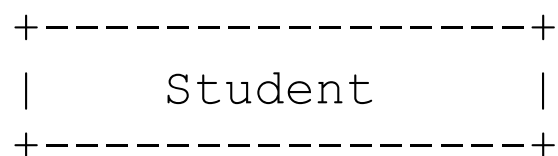
The following sequence represents the process of creating a room swap request:





7.4 Class / Component Diagram

The project contains eight main classes.



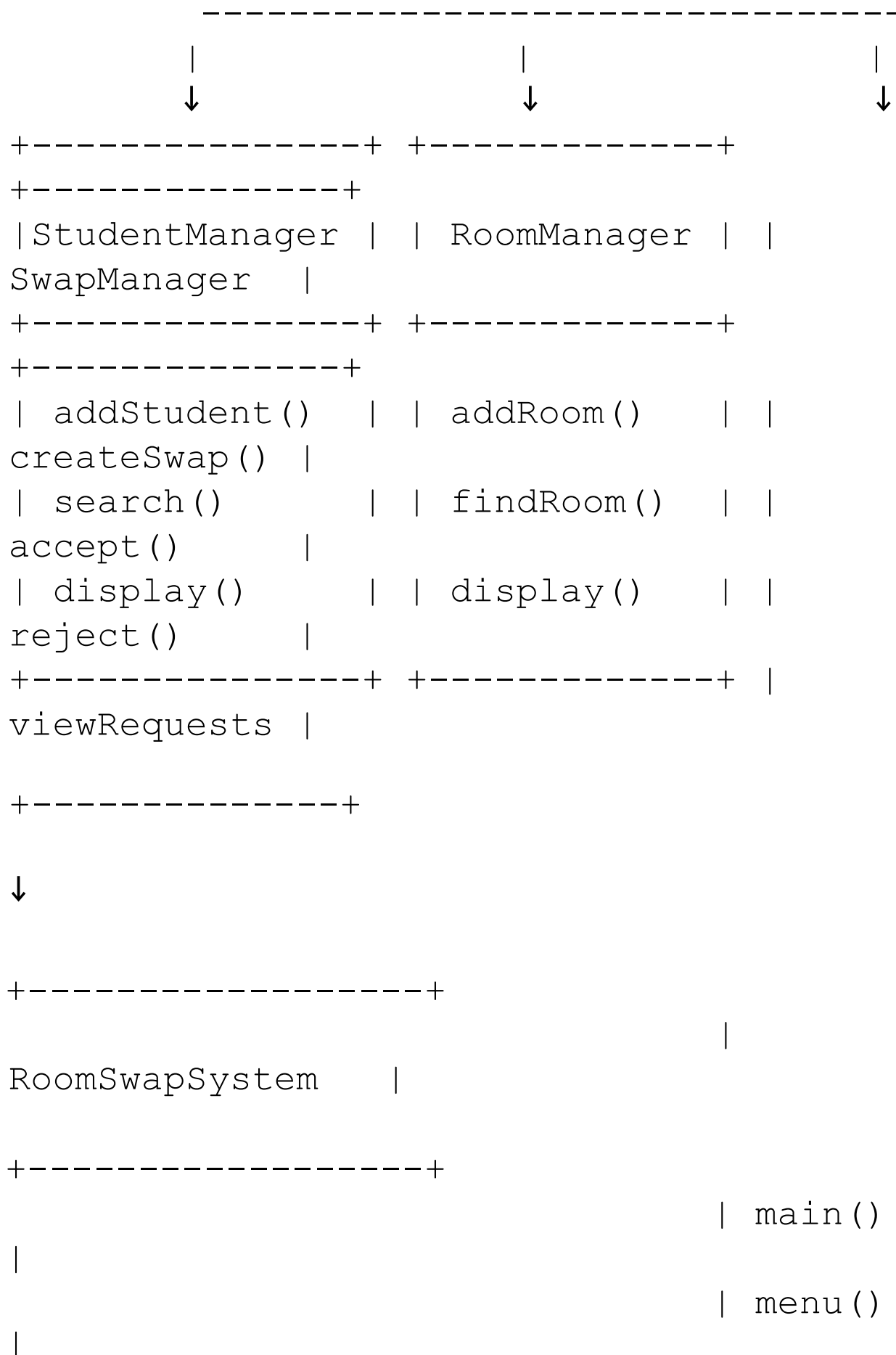
	studentId	
	name	
	email	
	currentRoom	
+-----+		

+-----+		
	Room	
+-----+		
	roomNumber	
	roomType	
	occupied	
+-----+		

+-----+		
	SwapRequest	
+-----+		
	requestId	
	student1	
	student2	
	status	
+-----+		

+-----+		
	Admin	
+-----+		
	adminId	
	name	
+-----+		





handling

|

input

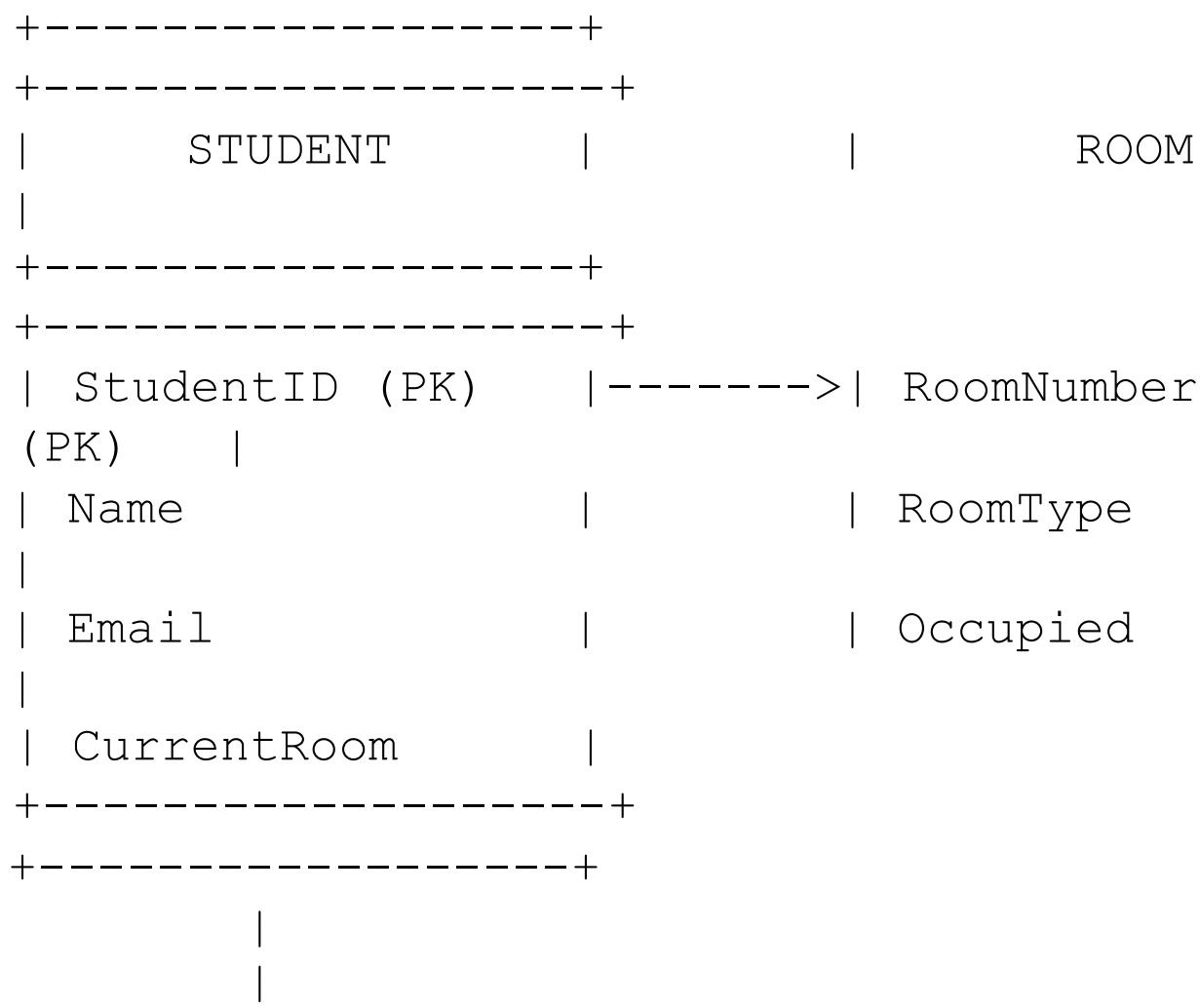
+-----+

7.5 ER Diagram

The current version of the project does **not use a database**. It stores information temporarily using Java `ArrayList`.

Therefore, an ER diagram is **not required for the actual implementation**.

However, a conceptual representation of the entities for a future database version can be shown as:





+-----+	
SWAP_REQUEST	
+-----+	
RequestID (PK)	
Student1	
Student2	
Status	
+-----+	

This structure can be converted into actual database tables in a future version.

8. Design Decisions & Rationale

Several design decisions were made while developing the project.

8.1 Java

Java was selected because it is suitable for implementing Object-Oriented Programming concepts such as classes, objects, encapsulation, and methods.

8.2 Console Interface

A console-based interface was selected because it keeps the application simple and allows the complete project to run directly from the command line.

8.3 Multiple Classes

The project is divided into eight classes instead of putting all functionality into one class.

This makes the project more modular and easier to understand.

8.4 Manager Classes

Separate manager classes were created for students, rooms, and swap requests.

This prevents the main class from becoming too large and keeps related operations together.

8.5 ArrayList

`ArrayList` was selected for storing students, rooms, and swap requests because it provides a simple way to store multiple objects dynamically.

8.6 No Database

A database was not included in the current version because the main focus of the project is Java programming and Object-Oriented Programming.

The project can be extended with database connectivity in the future.

8.7 Input Validation

Validation was included to prevent common errors and make the application more reliable.

9. Implementation Details

The project consists of the following classes:

Class	Responsibility
<code>Student.java</code>	Stores student details such as ID, name, email, and current room.

Room.java	Stores room number, room type, and occupancy status.
SwapRequest.java	Stores swap request details and request status.
Admin.java	Represents the administrator.
StudentManager.java	Handles student-related operations.
RoomManager.java	Handles room-related operations.
SwapManager.java	Handles room swap requests.
RoomSwapSystem.java	Contains the main method, menu, and overall program flow.

Main Menu

The application provides the following options:

===== ROOM SWAPPING SYSTEM =====

1. Add Student
2. View All Students
3. Search Student
4. View Rooms
5. Request Room Swap
6. View Swap Requests
7. Process Swap Request
8. Exit

Compilation

The Java files can be compiled using:

```
javac src/*.java
```

Execution

The application can be started using:

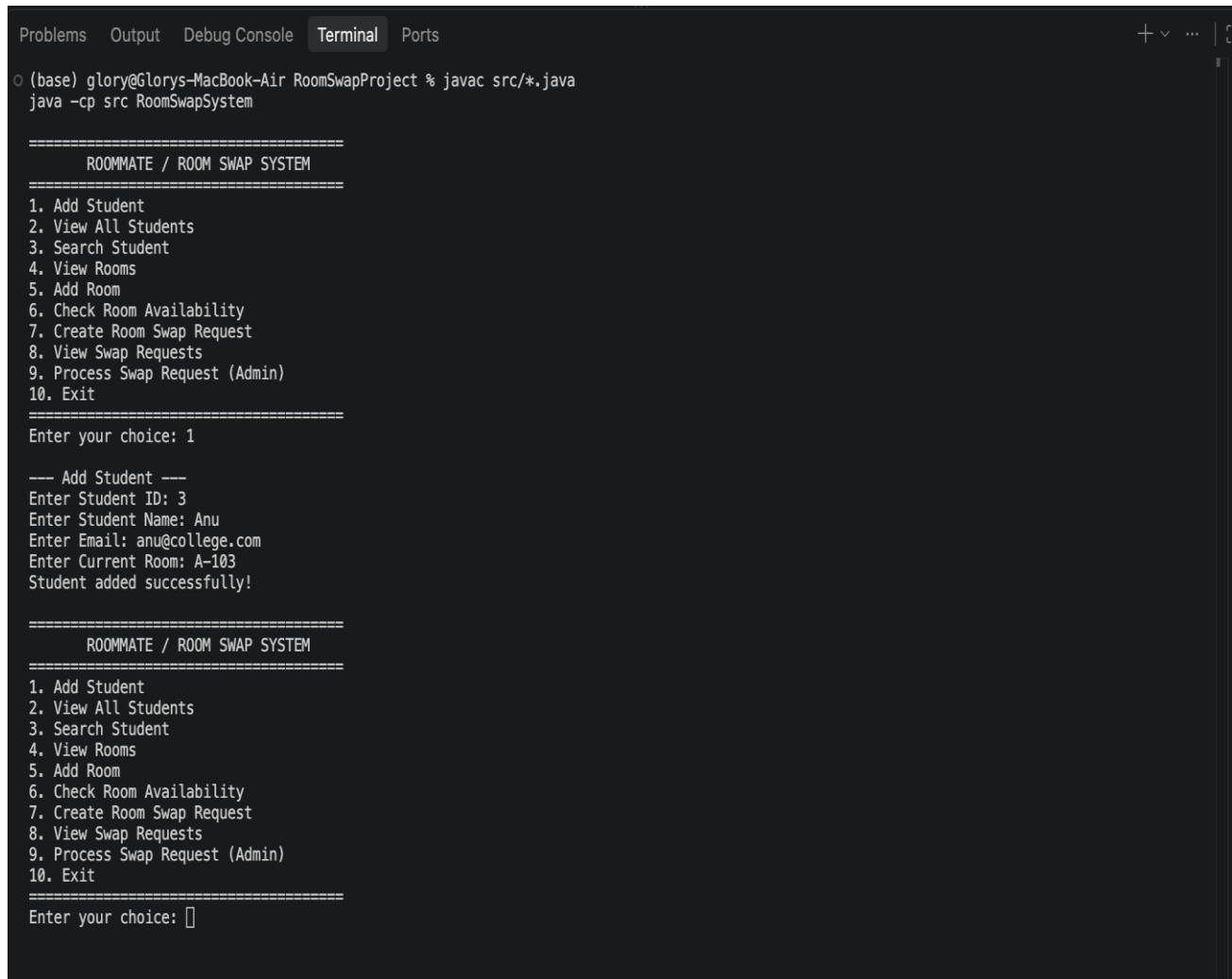
```
java -cp src RoomSwapSystem
```

OOP Concepts Implemented

The project demonstrates:

- Classes and objects
- Encapsulation
- Constructors
- Getters and setters
- Methods
- Object relationships
- ArrayList
- Conditional statements
- Loops
- Switch statements
- Input validation
- Error handling

10. Screenshots / Results



```
Problems Output Debug Console Terminal Ports
(base) glory@Glorys-MacBook-Air RoomSwapProject % javac src/*.java
java -cp src RoomSwapSystem

=====
ROOMMATE / ROOM SWAP SYSTEM
=====
1. Add Student
2. View All Students
3. Search Student
4. View Rooms
5. Add Room
6. Check Room Availability
7. Create Room Swap Request
8. View Swap Requests
9. Process Swap Request (Admin)
10. Exit
=====
Enter your choice: 1

--- Add Student ---
Enter Student ID: 3
Enter Student Name: Anu
Enter Email: anu@college.com
Enter Current Room: A-103
Student added successfully!

=====
ROOMMATE / ROOM SWAP SYSTEM
=====
1. Add Student
2. View All Students
3. Search Student
4. View Rooms
5. Add Room
6. Check Room Availability
7. Create Room Swap Request
8. View Swap Requests
9. Process Swap Request (Admin)
10. Exit
=====
Enter your choice: 
```

11. Testing Approach

Testing was performed using different valid and invalid inputs.

The main purpose of testing was to verify that each feature works correctly and that the system handles incorrect input without crashing.

Test ID	Test Case	Expected Result
---------	-----------	-----------------

T01	Add a valid student	Student is added successfully.
T02	Search for an existing student	Correct student information is displayed.
T03	Search for a non-existing student	Appropriate error message is displayed.
T04	View rooms	Room details are displayed correctly.
T05	Create a valid swap request	Swap request is created successfully.
T06	Student tries to swap with themselves	System rejects the request.
T07	Accept a pending request	Request is accepted and rooms are exchanged.
T08	Reject a pending request	Request status changes to Rejected.
T09	Process an already processed request	System prevents the operation.
T10	Enter an invalid menu option	Appropriate error message is displayed.

Testing Method

The project mainly uses **functional testing**.

Each menu option was tested individually using both valid and invalid inputs.

The output displayed by the system was then compared with the expected result.

12. Challenges Faced

Several challenges were faced during the development of the project.

12.1 Managing Multiple Classes

Dividing the project into multiple classes while keeping the code understandable was initially challenging.

12.2 Managing Student and Room Relationships

The system needs to correctly connect each student with their current room.

12.3 Room Exchange Logic

When a request is accepted, the rooms of two students need to be exchanged correctly without losing the original room information.

12.4 Input Validation

Different invalid inputs had to be handled, such as incorrect Student IDs, incorrect Request IDs, and invalid menu options.

12.5 Preventing Invalid Swaps

The system needed to prevent a student from creating a room swap request with themselves.

12.6 Request Status Management

Once a request is accepted or rejected, it should not be processed again.

13. Learnings & Key Takeaways

This project helped in understanding both Java programming and basic software design.

The major learnings include:

- Learned how to create and use classes and objects.
- Improved understanding of Object-Oriented Programming.
- Learned how encapsulation is implemented using private variables and getters/setters.

- Learned how constructors initialize objects.
- Learned how to use `ArrayList` to store multiple objects.
- Learned how different classes can communicate with each other.
- Learned how to divide a large problem into smaller modules.
- Learned how to create a menu-driven console application.
- Improved understanding of input validation.
- Learned how to handle invalid user operations.
- Understood the importance of testing different types of inputs.
- Learned how UML diagrams can be used to plan a software system before implementation.

14. Future Enhancements

The current system can be improved with several additional features.

14.1 Database Integration

A database such as MySQL can be added to permanently store student, room, and swap-request information.

14.2 File Handling

File handling can be used to save information so that data is not lost when the program is closed.

14.3 Login System

Separate login systems can be created for students and administrators.

14.4 Roommate Preference Matching

Students could enter preferences such as study habits, course, or other requirements, and the system could help find suitable roommates.

14.5 Advanced Search

Students could search rooms based on:

- Room type
- Availability
- Current occupants
- Other preferences

14.6 Notifications

The system could notify students when their swap request is accepted or rejected.

14.7 Admin Dashboard

An administrator dashboard could provide information about:

- Total students
- Occupied rooms
- Available rooms
- Pending requests

- Completed swaps

14.8 GUI / Web Application

The console application could later be converted into a graphical or web-based application.

15. References

1. Oracle Java Documentation – Java Programming Language and Standard Library documentation.
2. Oracle Java Tutorials – Classes, Objects, Constructors, Encapsulation, and Collections.
3. Java `ArrayList` Documentation – Used for managing collections of objects.
4. Java `Scanner` Documentation – Used for taking input from the command line.
5. Java Development Kit (JDK) Documentation.
6. Course learning materials and classroom notes related to Java and Object-Oriented Programming.
7. Course learning materials related to software design, system architecture, UML diagrams, and testing.